
```
basicstyle=, commentstyle=, keywordstyle=, frame=single, frameround=tttt,
breaklines=true, numberstyle=, stringstyle=, escapeinside=[**]
bashlanguage=bash
sqllanguage=sql
texlanguage=TeX
pythonlanguage=python
json morestring=[b]", stringstyle=, morekeywords=false,null,true
literate=À1 é1 è1 ô1 î1 ê1 à1 ù1 â1 ç1
defaultstyle leftmargin=25mm, rightmargin=25mm, innerleftmargin=0mm, innerrightmargin=0mm, innertop-
margin=1mm, innerbottommargin=1mm, roundcorner=10pt, linewidth=2pt, linecolor=black
```

Webamc 1.0.0 — A web interface for latex AMC MCQ

Sami Evangelista

March 19, 2026

Abstract

This document describes the installation and use of the webamc python package that allows to submit and view latex AMC MCQ through a web interface. It consists of:

- a command-line tool allowing to compile latex MCQ ;
- and a web interface allowing to load compiled MCQ in a database and view them.

Contents

1	Package installation	3
1.1	Dependencies installation	3
1.2	Webamc installation	3
2	Database and Web server configuration	4
2.1	Database creation	4
2.2	Database initialisation	4
2.2.1	Tables creation	4
2.2.2	CSV files loading	4
2.3	Web server execution and configuration	4
2.4	Apache configuration	4
3	File compilation and submission	5
3.1	Structuring tex files for webamc	5
3.1.1	Header tex file - _header.tex	5

1 Package installation

This section describes the installation procedure for the webamc package. This procedure has been realised on *Debian 13*.

1.1 Dependencies installation

We will first update the system. `sudo apt update` `sudo apt upgrade`

For a basic use of webamc you will need a python3 environment and optionally the git system (if you wish to install webamc from the development version). `sudo apt install python3 python3-pip python3-venv git`

If you install webamc on the web server side you will need the apache web server, the postgresql database manager, and the wsgi module so that our web server can execute our python scripts. `sudo apt install apache2 postgresql libapache2-mod-wsgi-py3 libpq-dev`

Now if you install webamc on the client side (i.e., to compile latex MCQ that you intend to load to the web interface) you will need a tex system able to compile tex files to pdf files: `sudo apt install texlive-latex-extra-utils`

1.2 Webamc installation

We will run python in a virtual environment `py3-venv` that we create here in `/home/webamc`: `whoamiwebamc cd /home/webamc python3 -m venv py3-venv`

We then activate the virtual environment. `source py3-venv/bin/activate`

We can finally install webamc either from the repository: `(py3-venv) git clone https://depot.lipn.univ-paris13.fr/evangelos/webamc` or from the last available release: `(py3-venv) pip3 install https://lipn.univ-paris13.fr/evangelos/webamc`

2 Database and Web server configuration

This section describes the database and web server configuration procedure. It has been realised on a *Debian 13* system and is intended for system administrators. It assumes that you have already installed the webamc package, following the instructions in Section 1.

2.1 Database creation

Once package has been installed, a database must be created. Right now the database cannot be created with the webamc package, so we will interact with the server using the `psql` client tool. First connect to the database server using `postgres` identity. `sudo -u postgres psql`

In the postgresql interpreter, create the database, a database user and grant this user all privileges on the created database: `create user webamc_u superpassword 'webamc_password'; createdatabase webamc; alterdatabase webamc owner to webamc_u;`

2.2 Database initialisation

The next step is to initialise the database: we have to create tables and put some initial data in these. This is done via the `webamc` tool provided by the package.

The configuration of the webamc command-line tool and web interface is done via a JSON file defining several configuration parameters (e.g., database access parameters). An example file can be found in the `examples/cfg` directory of the repository. Section ?? describes the different configuration parameters.

By default, the configuration file is loaded from the `.local/share/webamc/config.json` file in the home directory of the user.

2.2.1 Tables creation

First we need to initialise the database structure and create all its table: `webamc --config-file /path/to/my/config.json init` command normally silently terminates. An error will likely be due to a misconfiguration of the database access.

2.2.2 CSV files loading

The `webamc loadcsv` command can be used to load records in the database. To use it, simply type this command followed by the path of one (or more) CSV file(s). `webamc --config-file /path/to/my/config.json loadcsv *.csv`

The `webamc genexamples` command creates a bunch of example CSV files in the directory passed as argument. `mkdir csv webamc genexamples csv ls csv 00-usr.csv 01-cas_auth.csv 02-basic_auth.csv 03-tag.csv 04-grp.csv 05-usr_grp.csv 06-tbl.csv 07-admin.csv webamc --config-file /path/to/my/config.json loadcsv csv/*.csv`

For a basic setup, you will at least need to insert a user with administration privileges and an authentication mode so that she/he can access the web interface. CSV files can also be loaded from the web interface. Section ?? describes the structure of CSV files.

2.3 Web server execution and configuration

Once the webamc package is installed, you can simply launch webamc's web server via `uvicorn` and bind it to TCP port, e.g., 8008. `uvicorn webamc.www.app : app --port 8008`

You can also change set the `CONFIG_FILE` environment variable to specify the configuration file to load:

`CONFIG_FILE = /path/to/my/config.json uvicorn webamc.www.app : app --port 8008`

The `webamc.service` file in the root directory of the repository contains a minimal service specification to interact with the service via `systemd`.

2.4 Apache configuration

Now that our server listens on the 8008 port, we configure apache to redirect requests addressed to our server to this port. The first step is to enable the `proxy_http` module of apache:

`sudo a2enmod proxy sudo a2enmod proxy_http`

Then we have to add one line in the configuration of apache's virtual host to pass our server all requests on paths starting by, e.g., `/webamc`.

`ProxyPass /webamc http://localhost:8008/`

Then restart apache: `sudo systemctl restart apache2.service`

3 File compilation and submission

This section describes how the webamc command-line tool can be used to prepare, compile and submit MCQ that can be loaded via the web interface. It assumes that you have already installed the webamc package, following the instructions in Section 1.

3.1 Structuring tex files for webamc

MCQ submitted to the web interface may be of two forms:

- either a list of questions ;
- or a list of exercises, each exercise being a list of questions.

From now on, the term *component* will be used to denote either an MCQ, or an exercise, or a question. In addition to all components that are part of it, an MCQ or an exercise may have a statement, explaining, e.g., the context of all its components. This is the purpose of files `_mcq.tex` and `_exercise.tex` to contain this statements.

The following rules must be respected so that webamc can correctly compile your code:

- The content of an MCQ or an exercise must be placed in a single directory.
- To be recognised as an MCQ directory, the directory must contain an `_mcq.tex` file containing the MCQ statement even if it is empty.
- Likewise, to be recognised as an exercise directory, the directory must contain an `_exercise.tex` file containing the exercise statement even if it is empty.
- Questions must be placed in files of which the name is prefixed by `qst-`.

This is, for example, the directory structure of an MCQ containing five question files: `[style=defaultstyle] .1 some_mcq. .2 _header.tex. .2 _mcq.tex. .2 qst-aaa.tex. .2 qst-bbb.tex. .2 qst-ccc.tex. .2 qst-ddd.tex. .2 qst-eee.tex.`

This is another example of the directory structure of a two exercises MCQ (the first one with 2 question files and the second one with three question files): `[style=defaultstyle] .1 some_other_mcq. .2 _header.tex. .2 _mcq.tex. .2 exe-fst. .3 _exercise.tex. .3 qst-fst-1.tex. .3 qst-fst-2.tex. .2 exe-snd. .3 _exercise.tex. .3 qst-snd-1.tex. .3 qst-snd-2.tex. .3 qst-snd-3.tex.`

More complex examples may be found in the examples `examples/mcq` directory.

You may also submit directories that do not contain any MCQ but only question files. This is useful if you, for example, wish to record some questions in the database before creating question packs (see Section ??) that may contain these question.

Note that all the file names listed above (`_header.tex`, `_exercise.tex`, ...) are only default names and may be redefined through configuration files (see Section ?? for further details).

3.1.1 Header tex file - `_header.tex`

An `_header.tex` file may contain latex code that will be placed in the preamble of the latex document generated on-the-fly for each component of the directory. Typically, you will write in this file all `\usepackage{...}` instructions required to compile latex files in this directory. A directory inherits from any `_header.tex` contained in any of its ancestor directory. For example, let's consider this directory structure: `[style=defaultstyle] .1 some_dir. .2 _header.tex (content: \usepackage{packa}). .2 another_dir. .3 _header.tex (content: \usepackage{packb}). .3 _exercise.tex (content: This is a new exercise).` When compiling the exercise file `_exercise.tex`, the latex document compiled will contain the following code: `article packa packb This is a new exercise`